



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Fran Babić, Toni Brajko, Jakov Celin, Josip Klepec, and Iva Tadić.

Implementation examples are given in attached source code files.

Task Paradoks

Prepared by: Iva Tadić

Necessary skills: array, for loop

The goal of this task is to neatly and efficiently store the data given in the task so that it can be manipulated as easily as possible.

Since it is generally easier to work with numbers than with strings, let's represent our players with numbers 1 through 5, in the order they would play if Sonja were to start the round (i.e., Sonja is represented by 1, Viktor by 2, etc.). Similarly, we will represent the colors with numbers 1 through 4 (where red is "color 1", blue is "color 2", yellow is "color 3", and green is "color 4").

Paradoxes can occur in two ways — a card has already been played in the game and/or the player does not have that color in their hand.

We will track these states through two arrays:

- `se_pojavila_karta[x][y]` (*eng. card_has_appeared[x][y]*), which will be 1 if a card of color `x` and number `y` has already been played, and 0 otherwise.
- `ne_smije_koristiti_boju[x][y]` (*eng. not_allowed_to_use_color[x][y]*), which will be 1 if player `x` is not allowed to use color `y` for the remainder of the game, and 0 otherwise.

Note that at the beginning of the game, all values in both arrays will be 0 (no cards have been played and all players are allowed to use all colors).

We will now simulate the game, processing each input line by line (round by round).

We will track: the color of the round, which player is currently on turn, who is currently winning the round, and which card (color and number) is currently winning in that round. We will store paradoxes in a vector as we encounter them — chronologically.

At the start of each round, we set the first card played in the round as the winning card and the player who started the round as the current winner of the round.

For each subsequent card in the round, we first check if it causes a paradox by using the arrays mentioned above.

- If it is a paradox, we ignore the entire card throw (no new information is saved), and we record the paradox in our paradox vector.
(Note: we also skip updating the winner and proceed to analyze the next card throw in the round.)
- If it is not a paradox, we save the card as "appeared" (update the `se_pojavila_karta` array).
If the card played is not the color of the round, we conclude that the player on turn does not have a card of the round's color in hand, and we set `ne_smije_koristiti_boju[current_player][round_color] = 1`.

Furthermore, we compare the current card with the strongest card played in the round so far. If the current card is stronger, we update the winning player and set this card as the current winning card in the round. Now we examine the next card throw in that round...

After each round, we reset the variables for the winning player and the winning card. We repeat this process until we have processed the last round.



Finally, we count how many paradoxes we found (i.e., the number of elements in the vector where we stored the paradoxes) and print the paradoxes in order.

For implementation details, see the attached code.

Task Igre

Prepared by: Vito Anić

Necessary skills: dynamic programming (*Knapsack problem*)

Let's first solve the third subtask where $p_i = 0$. We can then simplify the problem: We have a backpack that can hold up to d kilograms of items. We have n items, each with its own weight and value. We want the total value of items stored in the backpack to be as large as possible. We can take each item more than once into the backpack. This is a classic example from dynamic programming known as the *Knapsack problem*. Let's briefly describe the solution.

The state will be $dp(x, k)$, where x indicates how many items we have considered so far, and k denotes how much capacity we have used so far. The transition will be:

$$dp(x, k) = \max(dp(x, k - 1), dp(x - 1, k), dp(x, k - t_x) + o_x)$$

In other words, we have three options: we either leave the k -th unit of capacity empty (perhaps we want to use it for a later item), we decide not to include the x -th item at all, or we include the x -th item (adding its value o_x to the total value) and fill the remaining capacity $k - t_x$ with other items (possibly the same ones again). By iterating over all x from 1 to n and over all k from 1 to d , we obtain the solution for this subtask. Those who want to know more, a detailed explanation of the knapsack problem and some of its variations can be found at this [link](#).

Now let's solve the entire problem. The issue we encounter is that before playing a game for the first time, we need to invest time in reading the rules. The current approach cannot separate the first time playing a game from subsequent times. How do we resolve this? We will add another variable to the state. The new state will be $dp(x, k, 0 \text{ or } 1)$, where x and k have the same meanings as before (x - which games we have considered so far, k - how much time we have invested so far), while the last bit will be 1 if we have learned the rules of game x and 0 if we have not. The transitions are then:

$$\begin{aligned} dp(x, k, 0) &= \max(dp(x - 1, k, 0), dp(x - 1, k, 1)) \\ dp(x, k, 1) &= \max(dp(x, k - p_i, 0), dp(x, k - t_i, 1) + o_x) \end{aligned}$$

If we haven't learned the rules yet, we can't do anything except spend the current time on previous games. If we have learned the rules, we could have done so in the last p_i minutes, or, provided that we knew the rules t_i minutes ago, we can play the game. The time complexity of the algorithm is $O(nd)$.

Task Različitost

Prepared by: Fran Babić

Necessary skills: ad-hoc, modular arithmetic

For the first subtask, it was sufficient to iterate over the first k pairs $a_i \oplus b_i$ because k is small enough that an $O(k)$ algorithm runs within the given time limit.

The second and third subtasks are solved by reducing the problem to a cyclic sequence of length m , $c_i = a_1 \oplus b_i$ ($c_i = a_i \oplus b_i$) for each $i = 1, 2, \dots, m$. We observe that the desired sum is $\sum_{i=1}^k c_i$, which can be relatively easily computed.

To achieve full points for the problem, it is useful to notice that this sum is independent across binary



digits, i.e.,

$$\sum_{i=1}^k a_i \oplus b_i = \sum_{j=0}^{\infty} \left(\sum_{i=1}^k a_{i,j} \oplus b_{i,j} \right) 2^j,$$

where $a_{i,j}$ denotes the binary digit corresponding to the power of 2^j , and similarly for $b_{i,j}$. This simplifies the problem by reducing it to arrays consisting of 0 and 1.

Now, for each element in the first array, we will sum all *bitwise exclusive-or* pairs in which that element appears. Due to the reduction to new arrays, this essentially boils down to summing all the elements it will pair with or summing the complements of those elements. The only remaining task is to determine these elements and compute their sum.

For this, a key observation is needed. Let $g = \gcd(n, m)$ be the greatest common divisor of n and m . For very large numbers k , the required elements of the second array, for a fixed element of the first array, form a cycle of length $\frac{m}{g}$. Additionally, the total number of distinct cycles (up to cyclic rotation) will be g . With an efficient implementation of this algorithm, achieving a complexity of $O((n + m) \log m)$, full points can be scored on this task.

Task Blistavost

Prepared by: Toni Brajko and Jakov Celin

Necessary skills: dynamic programming

Observation 1 *If we mark all points l_i , r_i , and point 0 on a number line, we can notice that there exists an optimal path for the guard to follow, which can be broken into segments with endpoints being these marked points. In the editorial, we will refer to these points as **important** points and sort them. The i -th important point in the sorted order will be denoted as T_i .*

The task is solved using dynamic programming. Since in the first subtask $n \leq 18$, let $dp[mask][j]$ represent the minimum time required to satisfy all conditions for which the bits in the mask are set and to end the path at point T_j . By fixing the mask, *important* point, and considering all conditions, we arrive at a solution with time complexity $O(2^n \cdot n^2)$.

In the second subtask, each range condition starts at point 1. It can be shown that, in this subtask, the guard will move either from left to right or go to the farthest point within the given ranges, wait there for a specific time if necessary, and then move left to the point 1. The proof is left as an exercise for the reader. Time complexity of the algorithm is $O(n \log n)$ due to sorting the points.

Observation 2 *If we record the last time each position is visited and save the list of positions as an array P , the guard's task is to design a path such that, for each condition of the residents, it holds that $\min(P[j]) \geq t_i$ for $l_i \leq j \leq r_i$.*

Observation 3 *If we observe the guard's path in reverse ("backwards"), the guard alternately expands the interval of visited points to the left and right. Within the interval of already visited points, the guard can move freely without needing to record the time of revisiting those points.*

Let $dp(l, r, 0)$ be the minimum time for array P to satisfy all resident conditions, considering only the points outside of the interval $(Tl, Tr]$. Let $dp(l, r, 1)$ be the minimum time for array P to satisfy all resident conditions, considering only the points outside of the interval $[Tl, Tr)$.

Using Observations 1 and 3, it is easy to show that from any state $dp(l, r, g)$, there are 4 states to which it can directly transition: $dp(l, r - 1, 0)$, $dp(l, r - 1, 1)$, $dp(l + 1, r, 0)$, $dp(l + 1, r, 1)$.

If we process intervals from larger lengths to smaller ones, the solution is obtained by taking the minimum value $dp(i, i, 0)$ for each index i of the important points. The time and memory complexity of the described



algorithm is $O(n^2)$. A direct implementation of the above algorithm scored 90 points due to the memory constraints of the task.

For a complete solution, it was necessary to observe that each state $dp(l, r, 0/1)$ transitions to states with intervals that are 1 shorter in indices l, r . With a careful implementation, the memory complexity can be reduced to $O(n)$, while the time complexity remains unchanged.

Task Trokuti

Prepared by: Josip Klepec

Necessary skills:

Problems of this type can be solved using a [flow](#) algorithm, which, if we set up the network properly, can help us find all $2 \cdot n$ triangles.

The original intended solution for this task was using a greedy algorithm with a exchange argument.

In the following few paragraphs, we'll describe the idea. We naively construct triangles one by one, using only vertices that we haven't used so far. Repeating this process gives us a certain number of triangles that may be less than n . More precisely, we'll get at least $\lfloor \frac{2}{3} \cdot n \rfloor$ triangles. In case we haven't obtained enough triangles, more than $3 \cdot n$ (half) of the vertices are still unused.

Looking at any $2 \cdot n$ disjoint triangles into which the graph can be partitioned, we conclude that none of them contains 3 previously unused vertices, because otherwise we could greedily make another triangle. Furthermore, since we have only $2 \cdot n$ triangles, and $3 \cdot n$ unused vertices, we conclude that there are at least n disjoint triangles that contain at least 2 previously unused vertices.

Looking at these n triangles and their third already used vertex, we conclude that there exist 2 triangles whose used vertices are in the same already greedily built triangle. What we can do is destroy that triangle and replace it with these new 2 triangles. This logic can be implemented in $O(n^4)$ time. The algorithm can be sped up to $O(n^3)$ by remembering pairs of unused vertices that can form a triangle for each already used vertex (also there can be an observation we won't need all of them for check part as we only need to build 2 triangles). All of which we can efficiently maintain by inserting and swapping triangles in each part of the algorithm, but this turned out to be unnecessary.

It turns out that the probability of getting at least n triangles in the greedy algorithm approaches 1 as n approaches infinity. It's also easy to show that the lower bound for the expected number of triangles we'll get is greater than n by looking at random graphs with 3 disjoint triangles.

The task could be solved by using a brute-force algorithm for small values of n or repeating the greedy algorithm many times, while for larger n we will always get n triangles.